Université de Strasbourg

IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

Introduction aux systèmes d'exploitation et à leur fonctionnement

CM1 : Linux, système de fichiers et commandes de base

Ali Ayadi

ali.ayadi@unistra.fr

BUT Informatique - S11

2026–2027

Plan

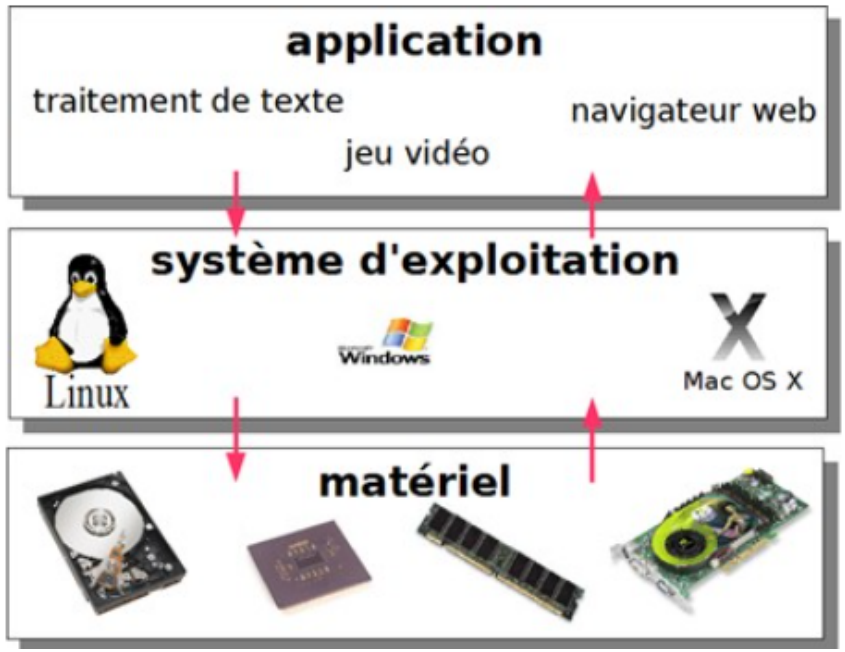
1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Plan

1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Qu'est-ce qu'un système d'exploitation

Définition : Un ensemble de logiciels chargé d'**établir une relation** entre les différentes **ressources matérielles**, les **applications** et l'**utilisateur**.



Exemple de SE : Windows, Unix (Linux, BSD, OSX), Mac OS X, Android, ...

Linux : un peu histoire[1]

- Unix :
  - 1970 : créé par ATT Corporation
  - 1977 : amélioré et distribué sous le nom de Berkeley Software Distribution (BSD)
  - 1980 : répandu dans les milieux universitaires
  - 1983 - 1990 : R. Stallman développe GNU, un S.E. libre compatible UNIX
- Minix :
  - 1987 : Minix, un S.E. clone d'Unix (à des fins pédagogiques)
- Linux :
  - 1991 : Linus Torvalds (étudiant finlandais) développe noyau de S.E., qui prendra le nom de "noyau Linux"
  - 1992 : Linux devenu officiellement un logiciel libre
  - 1993 : les premières "distributions" intégrant le noyau Linux apparaissent..

Une **distribution** = logiciels du projet GNU + noyau Linux
(e.g. Debian, Ubuntu basée sur Debian, Fedora, ...).

[1] un historique complet sur : https://www.levenez.com/

Où le trouve-t-on ?



Ordinateurs, serveurs, téléphone portables, consoles de jeu, robotique, automates industriels, appareil connecté, véhicules, ...

⇒ Chaque architecture matérielle demande un système d'exploitation adapté.

À quoi sert-il ?

Plusieurs missions, dont :

- **Accès** simplifier au matériel.
  Il dissimule :
  - les limitations physiques (processeurs, mémoires, . . .)
- **Gestion** des ressources (matérielles et logicielles) :
  - alloue les ressources aux applications qui le demandent
  - partage les ressources entre les applications
- **Protéger** l'intégrité du système

Pourquoi apprendre Linux en 2025 ?

- **Par curiosité** : comprendre ce qui se passe sous le capot (cœur d'un S.E.)
- **Universalité et utilité** : utile dans notre vie de tous les jours
  (ne serait-ce que pour installer votre maison connectée !)
- **Parce que ce sont des outils "conviviaux"** :
  - on comprend comment fonctionne la machine et les programmes
  - on peut réparer, modifier, créer par nous-mêmes (logiciel libre)
    $\rightarrow$ écrire des programmes qui utilisent efficacement les ressources
- **Open source** :
  - moins cher,
  - plus sécurisé,
  - plus portable (que des alternatives commerciales)
- **Être autonome** pour les autres enseignements en informatique

Plan

1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Le terminal Shell

shell est le nom donné au programme qui **gère la ligne de commande**.

Un programme qui sert d'**interface en mode texte** entre le noyau et l'utilisateur.

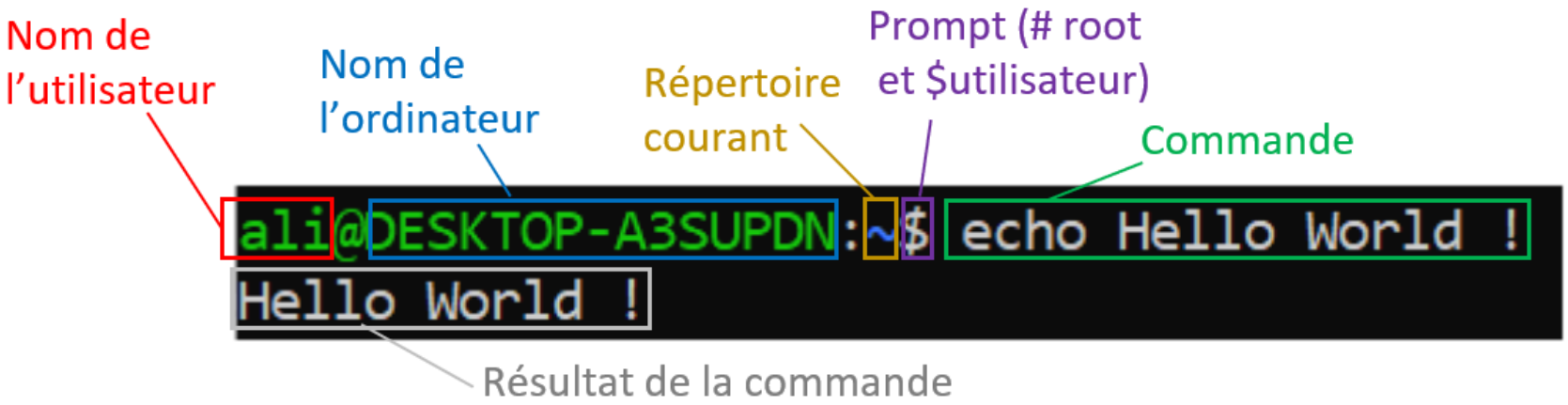
Interpréteur de commandes textuel et un **langage de programmation** fourni de manière standard par tous les UNIX

Les différents shells :

- `sh`, Bourne Shell, interpréteur historique des premiers UNIX de Steve Bourne
- `bash`, Bourne again shell, version étendue du bourne shell et par défaut sur linux
- `ksh`, `zsh`, `dash`, ..., plus ou moins vieille famille compatible avec Bourne
- `csh`, `tcsh`, famille non compatible avec Bourne, peu utilisés

```
ali@DESKTOP-A3SUPDN:~$ ls /bin/bash
/bin/bash
ali@DESKTOP-A3SUPDN:~$ echo $SHELL
/bin/bash
```

Fonctionnement de la ligne de commande



1. Affichage de l'**invite** de commande (**prompt**) : `ali@DESKTOP-A3SUPDN:~$`
2. Attente d'une commande, suivie de la touche Enter
   `ali@DESKTOP-A3SUPDN:~$ echo Hello World !` Enter
3. La commande s'exécute : est-ce une commande correcte ?
   - si oui, on l'exécute, et on attend que cette exécution se termine !
   - sinon, on affiche un message d'erreur

   Ici, la commande est correcte, donc exécution :

   → affichage de : Hello World !

Remarque : exécuter une commande signifie (en général) démarrer un programme chargé d'exécuter cette commande.

Commande : syntaxe messages d'erreur

Les commandes Unix sont toujours de la même forme :

```
nom_de_commande [options] [arguments]
```

c.-à-d. nom de commande suivi de 0 ou plusieurs options et/ou arguments

Exemples :
```
cp tp1.sh tp2.sh    # commande cp suivie de 2 arguments
mkdir TP            # commande mkdir suivie de 1 argument
ls                  # commande ls suivie d'aucun argument
ls -l TP1.txt       # commande ls suivie de 1 opt. et 1 arg.
```

Lorsqu'une commande est incorrecte, le shell ou la commande fournit un message d'erreur, différent selon la nature de l'erreur :

- <nom de commande> incorrect (message du shell)
  `-bash: salut : commande introuvable`
- <arguments> incorrects (message de la commande)
  `cp: missing file operand`
- la commande ne peut s'exécuter correctement
  `touch: cannot touch 'S11': Permission denied`

Raccourcis utiles

- Lancer un terminal : Ctrl + Alt +T
- Historique
  - Flèches ↓ et ↑ pour défiler les commandes dans l'historique
  - history affiche les dernières commandes que nous avons tapées
- Rechercher une commande : Ctrl +R
- Ctrl +l, clear et reset : pour faire le ménage (réinitialiser votre terminal)
- Complétion avec ⟺ (moins de choses à taper et de fautes de frappes)
- Tuer le processus courant (au cas où ça frise) : Ctrl +C
- Fermer le terminal : Ctrl +D
- Déplacer le curseur au début d'une ligne : Ctrl +A
- Déplacer le curseur à la fin d'une ligne : Ctrl +E

Plan

1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Système de fichiers

Un système de fichiers (**ext4** pour Linux) est une façon d'organiser et de stocker une arborescence sur une mémoire de masse (e.g. disque dur).

Sous Linux, "Tout est fichier" (programmes, documents, texte, fichiers de configuration, périphériques E/S, ...)
$\rightarrow$ Ces fichiers sont regroupés dans une arborescence unique ("/").

Un fichier est la réunion de :

- un seul *inode*
- un ou plusieurs *noms*
- un *contenu*, c'est-à-dire un ensemble ordonné d'octets
- un *propriétaire*
- des *horloges scalaires* (création, dernier accès, dernière modif)
- des *droits d'accès* (lecture, écriture, exécution)

Attention : Chaque fichier a un seul **inode** (numéro d'inode ou identifiant), même s'il peut avoir plusieurs **noms** (chacun de ceux-ci fait référence au même inode). Chaque nom est appelé **lien**.

Système de fichiers

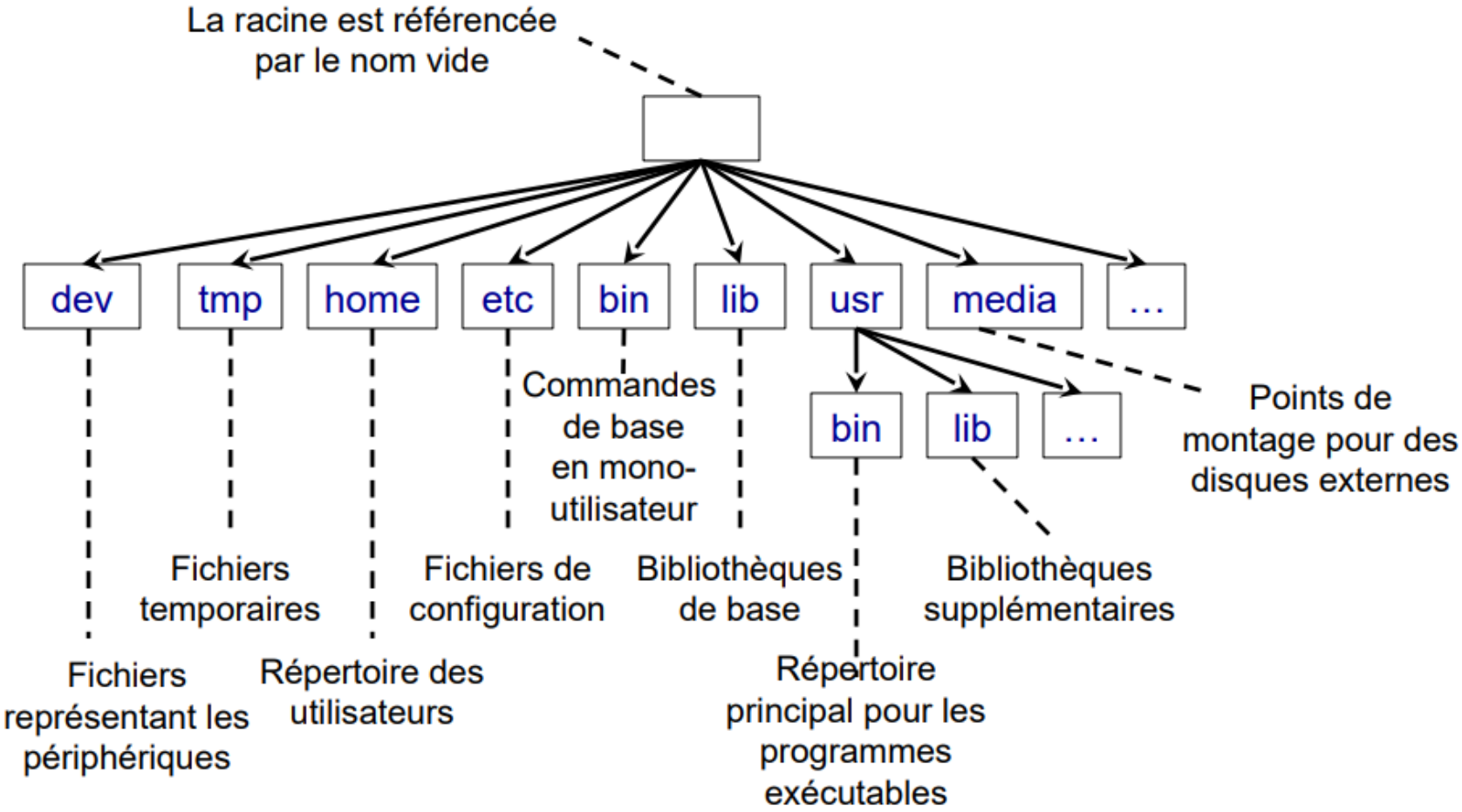
Problème : comment retrouver facilement un fichier parmi des centaines de milliers ?

Solution : en rangeant les fichiers dans des **répertoires** (dossiers).

- En *bash*, le séparateur de répertoires est le caractère **/**
- Un chemin s'écrit sous la forme **/home/ali/Documents/rapport.txt** qui référence :
  - **/**: La racine du système de fichiers.
  - **home**: Répertoire à la racine.
  - **ali**: Sous-répertoire de `home`.
  - **Documents** : Sous-répertoire de `ali`.
  - **rapport.txt** : Fichier se trouvant dans `Documents`.

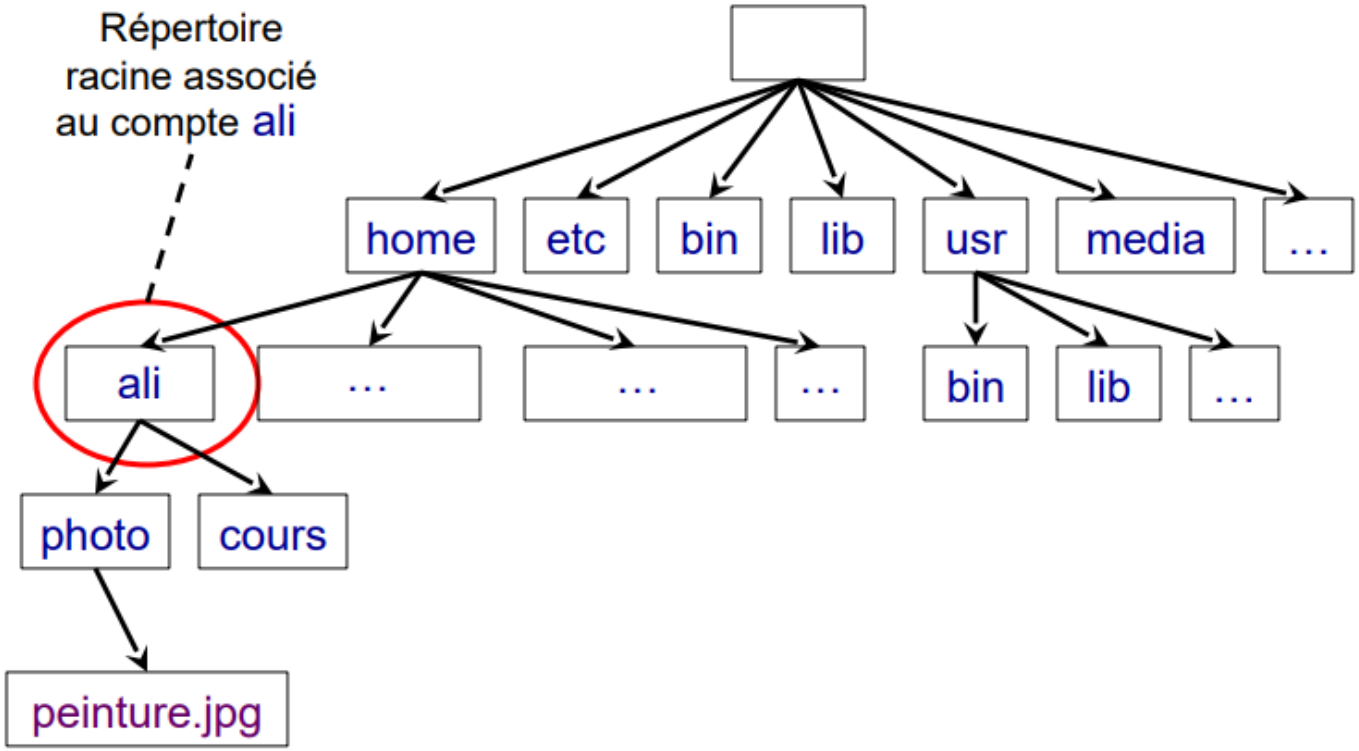
Arborescence standard des SE UNIX

La plupart des systèmes d'exploitation Unix (GNU/Linux, BSD, MacOS, ...) utilisent une arborescence de base standardisée. man hier



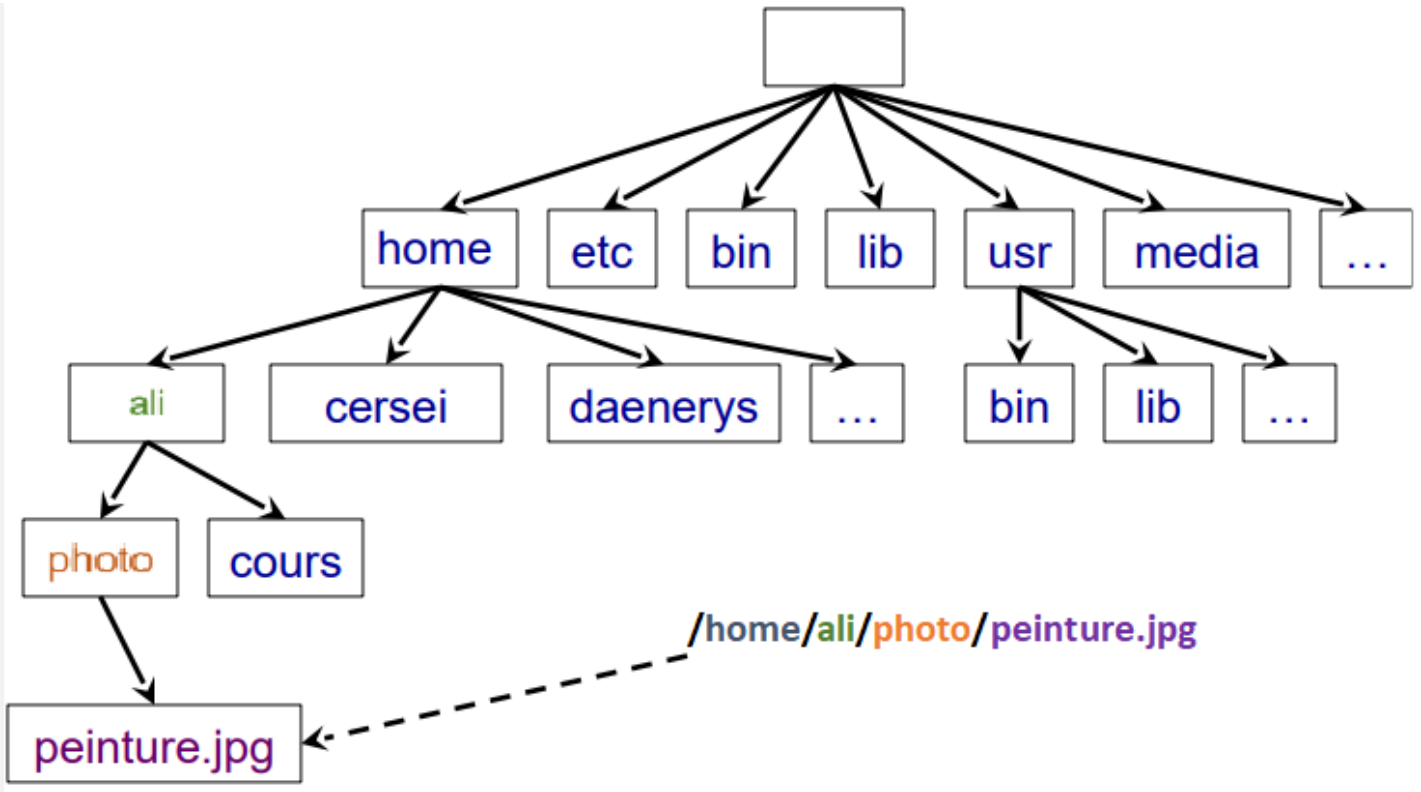
Arborescence standard des SE UNIX

Chaque utilisateur possède un *répertoire personnel* (de connexion ou **home directory**)
Accessible via la variable `HOME` (ou aussi ∼). `echo $HOME`

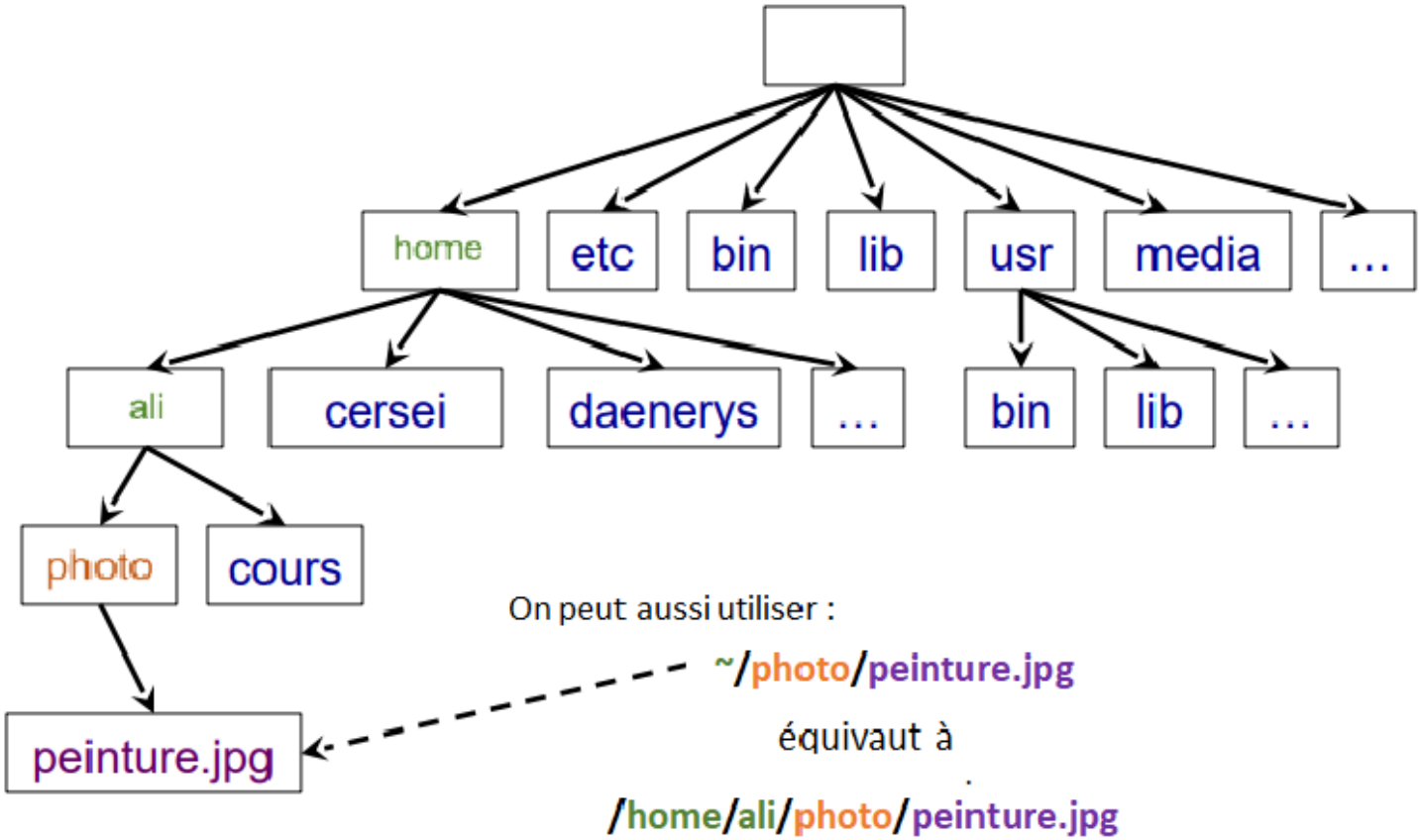


Chemin absolu (1/2)

Un **chemin absolu** : chemin dans l'arborescence depuis la racine (/) jusqu'à l'objet recherché (fichier ou répertoire) exprimé par une liste de répertoires (séparé par des /).



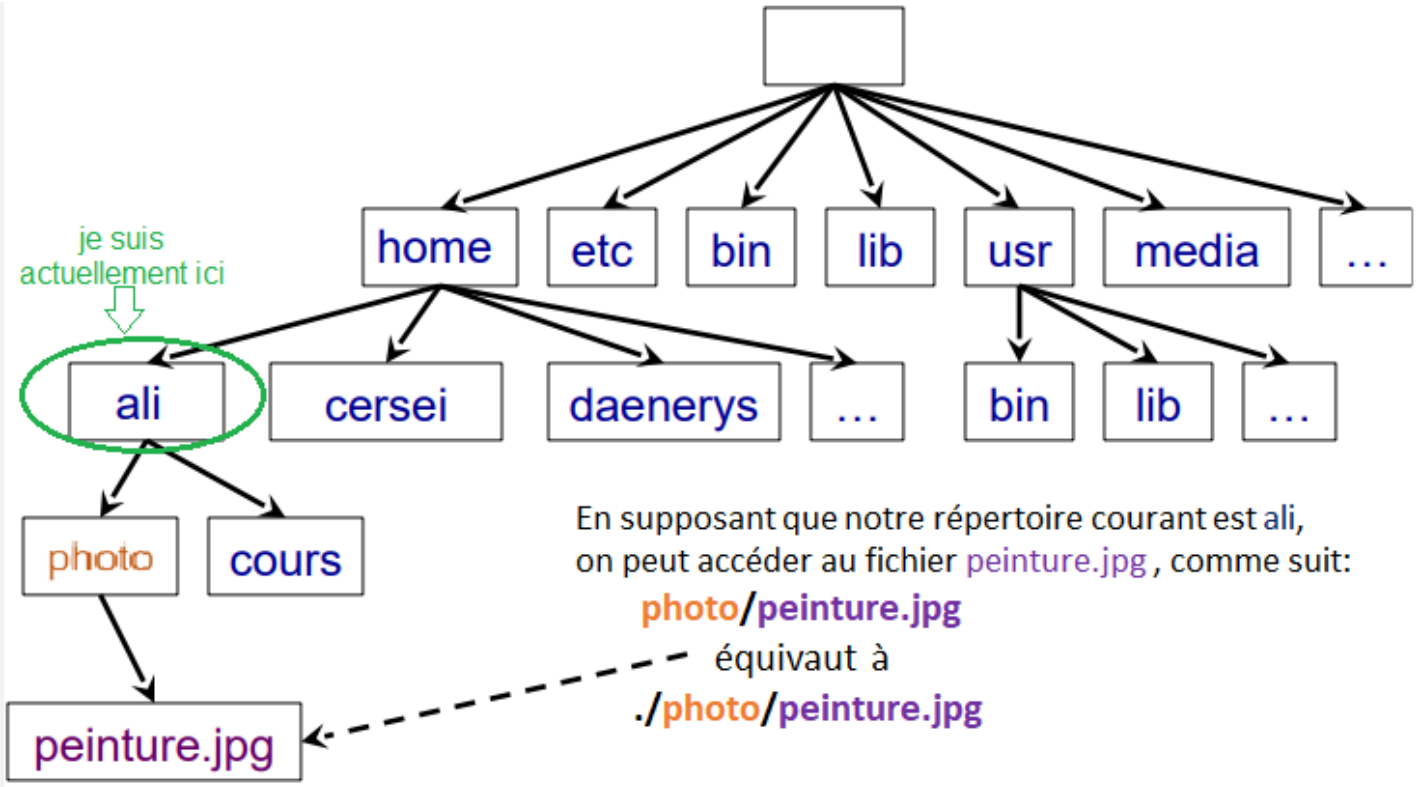
Chemin absolu (2/2)



Attention : Un utilisateur peut utiliser $\sim$ comme raccourci pour son répertoire de connexion.

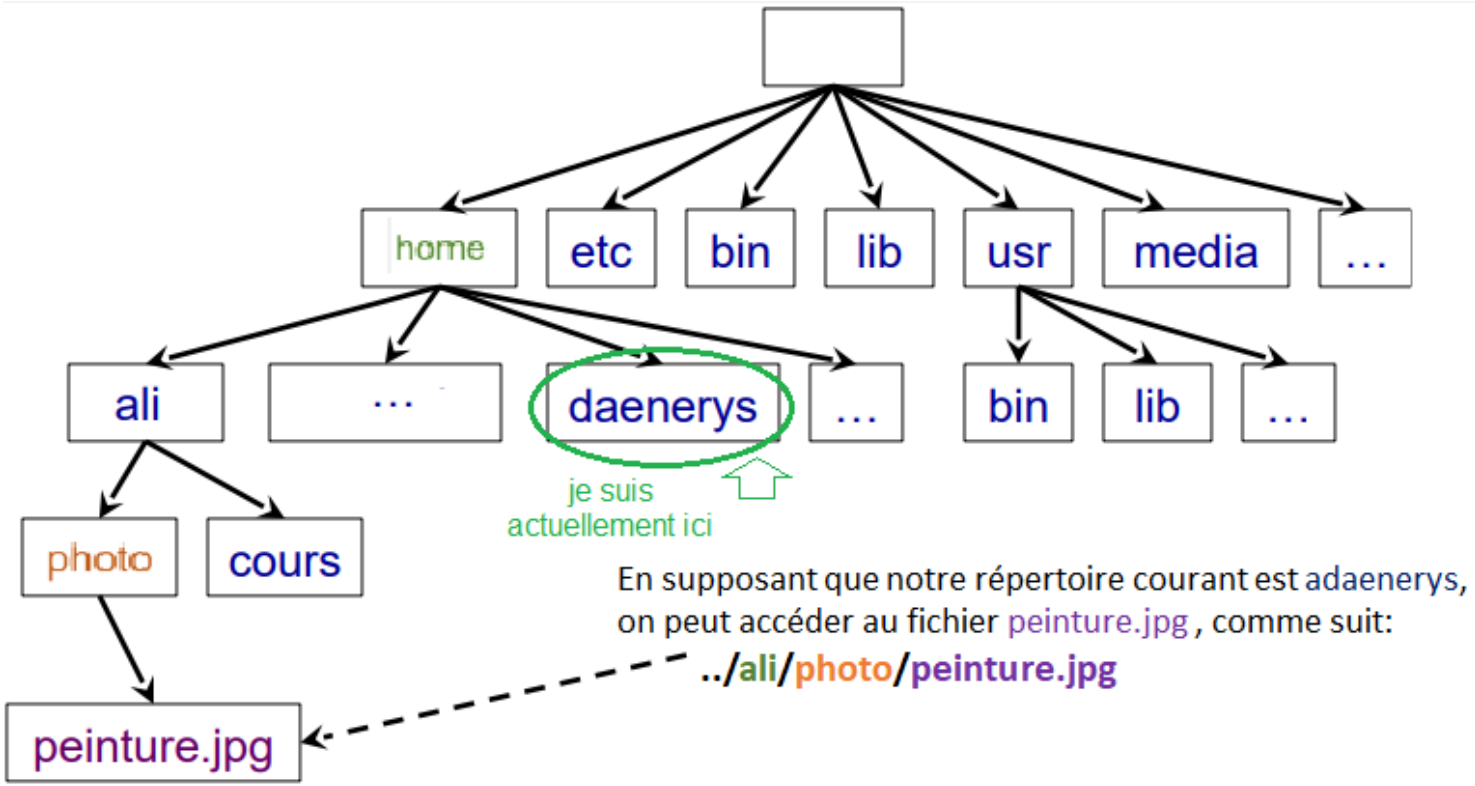
Chemin relatif (1/2)

Un **chemin relatif** : chemin dans l'arborescence depuis le répertoire de travail jusqu'à l'objet recherché (fichier ou répertoire) exprimé par une liste de répertoires (séparé par des /).



Attention : Chaque répertoire possède un fichier nommé **.** s'auto-référençant.

Chemin relatif (2/2)



Attention : Chaque répertoire possède un fichier nommé **..** référençant son parent.

Pour résumer : chemin d'accès

Tout objet de stockage (fichier ou répertoire) est identifié par sa place dans l'arborescence.

Les conventions suivantes (POSIX) s'appliquent :

- La **racine** est représentée par /
- Les répertoires sont **séparés** par des /
- Tout chemin qui **ne commence pas** par / est **relatif** au répertoire courant
- Le répertoire **courant** est symbolisé par .
- Le répertoire **parent** est symbolisé par ..
- Le répertoire **home** est symbolisé par ~

Nom de fichier

- Unix différencie les majuscules des minuscules !

  →`fichier.txt` et `Fichier.txt` sont deux fichiers différents.

- Unix se fiche des extensions, il ne détermine pas le type du fichier.
  Il s'agit juste d'une "bonne pratique" pour classer les fichiers.
  On peut avoir des fichiers sans extensions.

- Éviter les espaces, caractères spéciaux et accents dans les noms de fichiers :

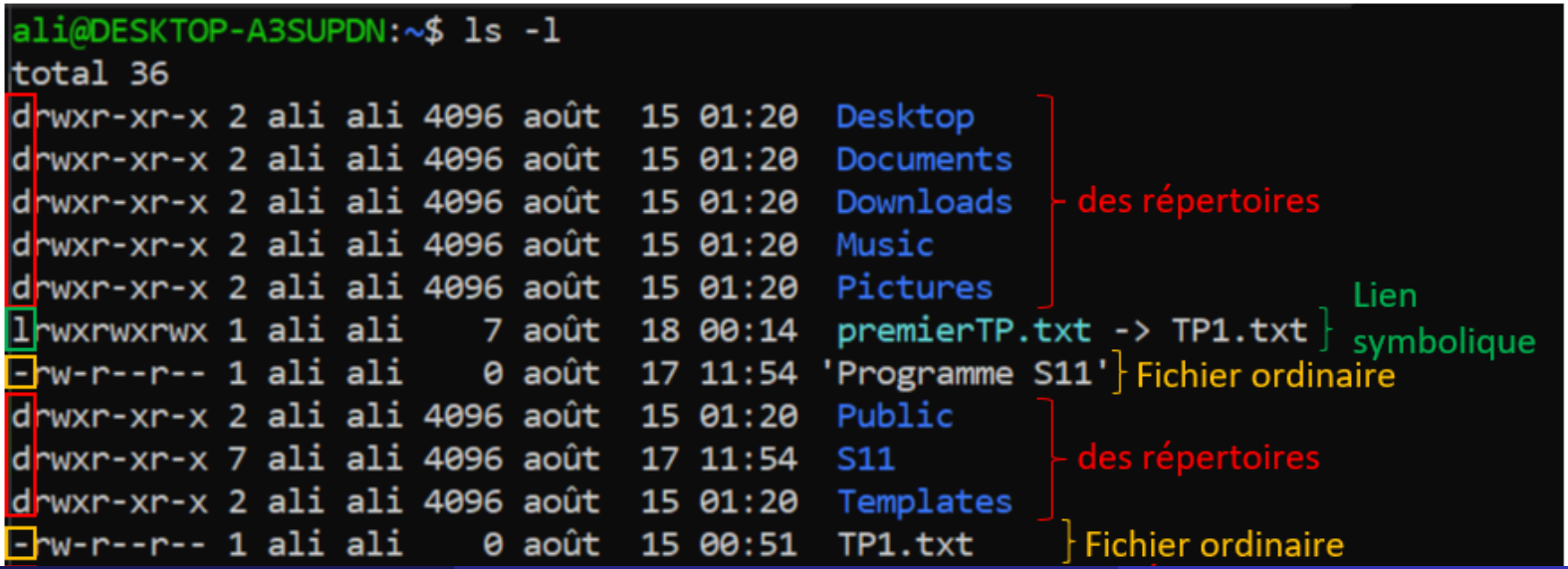
  - difficile à gérer dans les scripts SHELL,
  - pas portable, e.g. encodage (isolatin, UTF8, ...) des caractères accentués

Exemples : `rapport final.pdf` → `rapport_final.pdf`
`fichier_éxécution.sh` → `fichier_execution.sh`

Les types de fichiers

Dans LINUX, il existe différents types de fichiers :

- Les fichiers ordinaires (-)
- Les répertoires (d)
- Les périphériques (Devices)
  - Les périphériques bloc (b) (disques durs, lecteurs de CDROM)
  - Les périphériques caractère (c) (souris, clavier, ports)
- Les tubes (pipes) (p)
- Les sockets (s)
- Les liens (l)



Les liens

Il existe deux sortes de liens : le lien *physique* et le lien *symbolique*.

Un lien permet d'accéder à un même fichier (ou un répertoire) sous **plusieurs noms** (ou à partir de **plusieurs répertoires**).

Liens physiques

Un **lien physique** permet de donner plusieurs noms/chemin d'accès à un même fichier en pointant sur un numéro d'inode de fichier.

Le lien physique ne peut adresser que des fichiers.

Syntaxe : ln /fichier_cible /nom_du_lien

Exemple : ln /TP1.txt /firstTP.txt

Modifier le contenu du lien physique → Modifie en réalité le contenu de l'inode sur lequel il pointe (fichier source).

La suppression du fichier source, → casse le lien, mais ne supprime pas physiquement l'autre fichier.

```
ali@DESKTOP-A3SUPDN:~$ ls -i TP1.txt firstTP.txt
41146 firstTP.txt  41146 TP1.txt
```

⇒ les différents liens physiques **pointent vers un même numéro** d'index.
⇒ Ils associent plusieurs fichiers à un même espace sur le disque, les fichiers restant indépendants.

Lien symbolique

Un **lien symbolique** est un chemin alternatif (un *alias* ou un *raccourci*) pour le fichier d'origine.

Le lien symbolique peut adresser des fichiers et des répertoires.

Syntaxe : ln -s dossier_cible nom_du_lien

Exemple : ln -s /TP1.txt /premierTP.txt

permet d'attribuer un autre chemin d'accès à un fichier en pointant sur un nom de fichier.

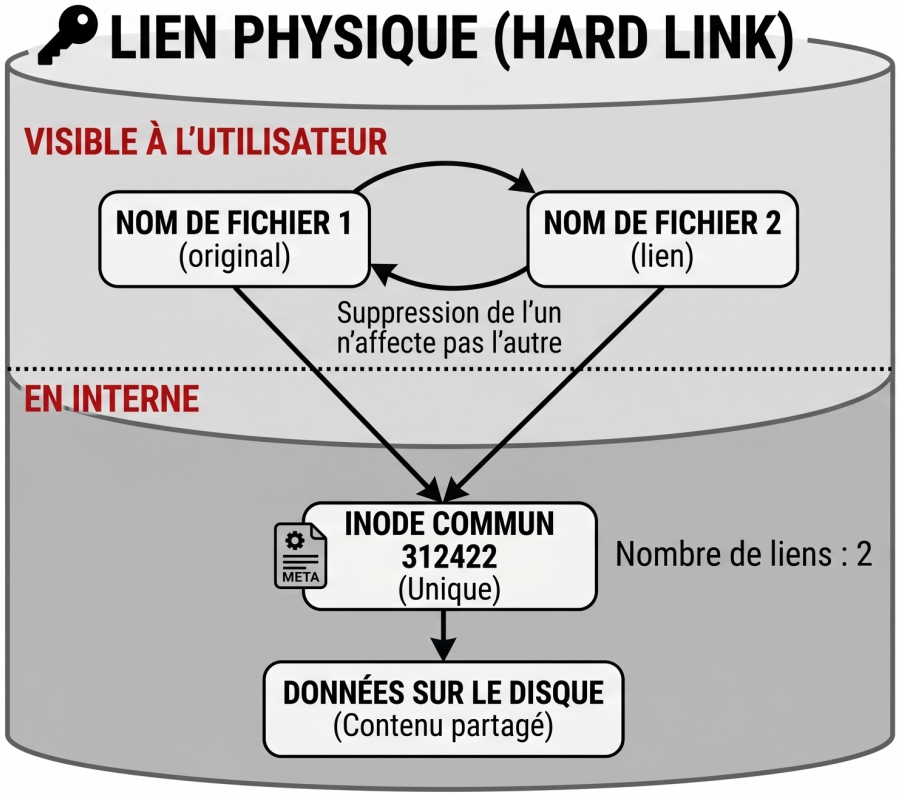
Lorsque le fichier d'origine est supprimé, le lien symbolique devient invalide.

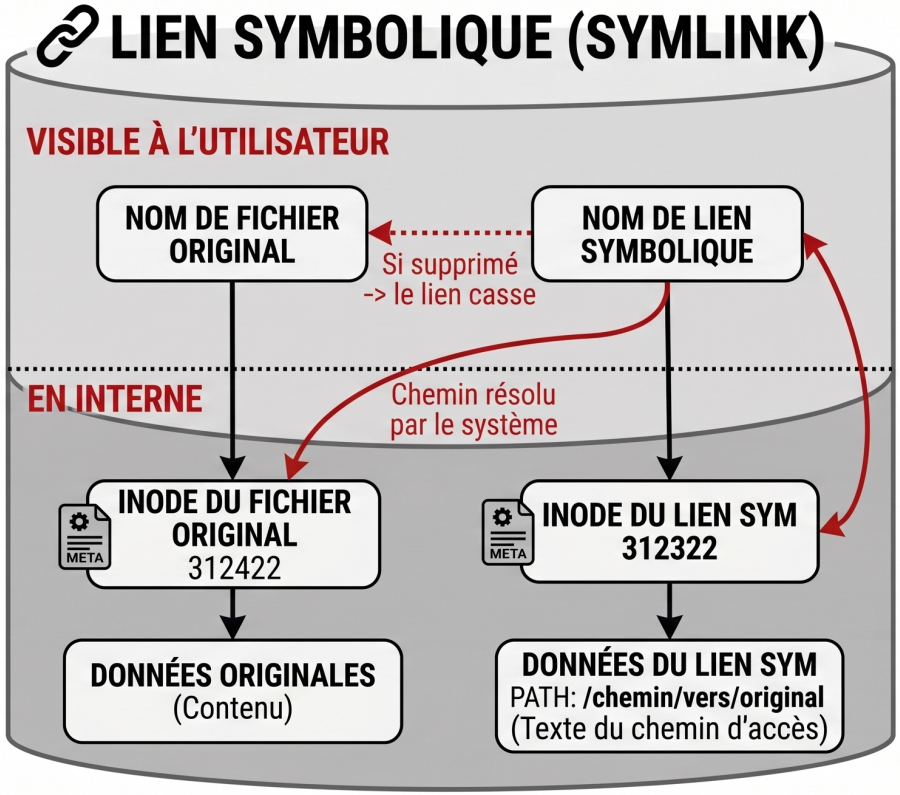
Le chemin relatif et le chemin absolu sont autorisés dans les liens symboliques.

```
ali@DESKTOP-A3SUPDN:~$ ls -i TP1.txt premierTP.txt
122742 premierTP.txt   41146 TP1.txt
```

⇒ les deux fichiers **pointent vers deux numéros d'inode différents**.

Lien physique VS lien symbolique





Plan

1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Droits d'accès : un système multi-utilisateurs

- Linux est un système **multi-utilisateurs** : plusieurs utilisateurs peuvent utiliser le système *simultanément*, ils partagent toutes les ressources (processeur, disque, mémoire, ...).

- $\rightarrow$ les permissions des utilisateurs doivent être **restreintes** afin de garantir l'intégrité de leurs données et du système lui-même.
  e.g. éviter que :
  - votre petit frère détruise vos fichiers
  - vos parents lisent vos courriers
  - votre petite sœur administre la machine

- Pour le système de fichiers, cela se traduit par la notion de droits d'accès.

- Toute opération sur un fichier est soumise à droits d'accès
  Message d'erreur "Permission non accordée"

Droits d'accès : catégories d'utilisateurs

Unix distingue **3** catégories d'utilisateurs :

- User (u) : Désigne la personne qui a créé le fichier/répertoire, c'est le **propriétaire** (peut être modifié par la suite).
- Group (g) : Désigne les **membres du groupe d'utilisateurs**.
  Exemples :
  - prof, étudiants, ... pour un établissement d'enseignement,
  - direction, comptabilité, infographie, ... pour une société.
- Others (o) : Désigne tous les **autres utilisateurs**.

Chaque catégorie possède des types d'accès aux fichiers/répertoires.

Droits d'accès : types de permissions

On distingue **3** types de permissions :

- r : **read** → permission de lecture
  - Si *répertoire*, **consultation** de ses entrées (c.-à-.d, `ls` autorisé)
  - Sinon, **consultation** du contenu du *fichier*
- w : **write** → permission d'écrire
  - Si *répertoire*, droit de **création**, de **renommage** et de **suppression** d'une entrée dans le *répertoire*
  - Sinon, droit de **modification du contenu** du *fichier*
- x : **execution** → permission d'exécuter
  - Si *répertoire*, droit de **traverser** r (c.-à-.d., `cd` autorisé)
  - Sinon, droit d'**exécution**

Pour visualiser les permissions sur les fichiers/répertoires : `ls -l`

type	u	g	o	#lnk	owner	grp	size	last update	name
d	rwx	r-x	r-x	10	ali	prof	4096	Sep 11 2023	cours
-	rwx	rwx	rwx	1	ali	prof	6770	Aug 1 2023	img.jpg
-	rwx	- - -	- - -	1	ali	prof	2001	Jul 6 2023	tp1.txt

Sémantique des permissions (fichier)

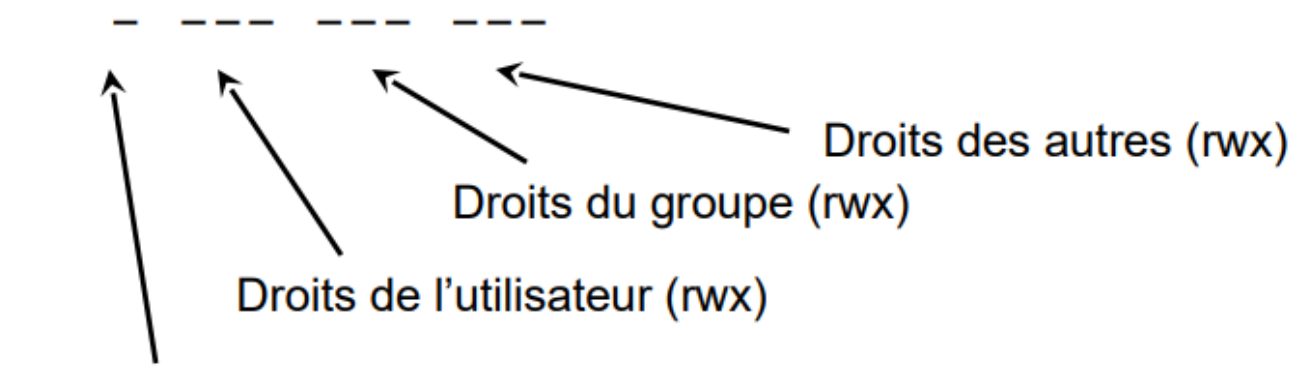
Fichier	r	Autorise la consultation - affichage - copie
	w	Autorise la modification - modification du contenu - **ne permet pas la suppression du fichier !** - **ne permet pas le changement de nom !**
	x	Autorise l'exécution - le fichier doit être un programme

Sémantique des permissions (répertoire)

Répertoire	r	Autorise la consultation - affichage du contenu (ls)
	w	Autorise la modification - ajout de contenu (e.g., les fichiers et les sous-répertoires) (touch, mkdir, ...) - suppression de contenu (rm et rmdir) - renommage de contenu (mv)
	x	Autorise la traversée - utilisation dans un chemin d'accès à une entrée (fichier/répertoire) (cd)

Droits d'accès : consultation

Format de sortie de ls –l :



Type du fichier :
d : répertoire
l : lien symbolique
- : fichier ordinaire

```
$ ls -l fichier
- rwx r-- --- fichier
$
```

Droits d'accès : Modification avec des lettres (1/2)

De manière symbolique

La commande `chmod` (change mode) permet de changer les droits.

Syntaxe : `chmod` *droit cible*

cible : fichier ou répertoire

droit : [u g o a] [+ - =] [r w x]

Catégories : user (u), group (g), other (o) ou all (a) (a = all = ugo)

Opérations : ajout (+), retrait (-), affectation (=)

Permissions : lecture (r), écriture (w), exécution (x)

Exemple :

- retirer les droits d'écriture et d'exécution pour le groupe et les autres sur le fichier *img.jpg*

```
chmod go-wx img.jpg
```

- ajouter les droits de lecture et d'écriture pour le groupe sur le fichier *tp_unix.txt*

```
chmod g+rw tp_unix.txt
```

Droits d'accès : Modification en octal (2/2)

De manière octale

La commande `chmod` (change mode) permet de changer les droits.

Syntaxe : `chmod` *droit cible*

cible : fichier ou répertoire

droit : 3 chiffres : le 1er $\rightarrow$ user, le 2ème $\rightarrow$ group, et le 3ème $\rightarrow$ other
chaque chiffre représente un "groupement" de droits (pour user, group et other) et à chaque droit correspond une valeur :

- **r** (read) = 4
- **w** (write) = 2
- **x** (execute) = 1
- \- = 0

Exemple :

- donner au propriétaire tous les droits, aux membres du groupe et aux autres les droits de lecture et d'accès sur le dossier *S11*

```
chmod 755 S11
```

Droits d'accès : représentation des droits d'accès

Représentation des droits d'accès dans les deux modes :

Correspondances de représentation des droits			
Droit	**Valeur alphanumérique**	**Valeur octale**	**Valeur binaire**
aucun droit	`---`	0	000
exécution seulement	`--x`	1	001
écriture seulement	`-w-`	2	010
écriture et exécution	`-wx`	3	011
lecture seulement	`r--`	4	100
lecture et exécution	`r-x`	5	101
lecture et écriture	`rw-`	6	110
tous les droits (lecture, écriture et exécution)	`rwx`	7	111

Plan

1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Rappel : syntaxe générale d'une commande

nom_commande [-option(s)] [argument(s)]

- La commande peut être suivie d'**options** ou d'**arguments** séparés par des espaces.
- Les options sont précédées d'un " - " contrairement aux arguments

Exemple : Lister des fichiers ls

```
ls [-altrR] [Fichiers/Répertoires...]
    -a : (all) tous les fichiers, même cachés
    -l : (long) lister au format long
    -t : (tri) lister en triant par date
    -R : (recursive) lister récursivement dans les répertoires
```

ls -l ; ls -ltr ; ls -R -l /usr ; ls -al . ; ls -lR /etc

Où trouver de l'aide

La commande `man` contient entre autres la description des commandes UNIX.

Exemple : description de la commande ls

```
man ls
```

Quelques séquences de touches utiles :

- `/<motif>` pour la recherche (puis `p` et `n`) pour précédent et suivant
- `Ctrl-b` et `Ctrl-f` pour reculer et avancer
- `q` pour quitter

Sur les repertoires

`pwd` : affiche le répertoire courant (*print working directory*).

`cd` : change de répertoire courant (*change directory*).

- `cd ..` remonte au répertoire père
- `cd /` va dans le répertoire racine
- `cd` va dans le répertoire principal (*home*) de l'utilisateur courant (équivalent de `cd ~` ou `cd $HOME`)
- `cd -` va dans le répertoire exploré précédemment

`mkdir` : créé un répertoire (*make directory*).

`rmdir` : `rmdir rep` supprime un répertoire s'il est vide (*remove directory*).

Sur les fichiers

`ls` : liste les fichiers et répertoires (*list*).

`cp` : copie des fichiers et répertoires (*copy*).

`rm` : supprime des fichiers et répertoires (*remove*).

`mv` : déplace et/ou renomme un fichier ou un répertoire (*move*).

`touch` : créé un fichier vide (ou change la date de dernier accès d'un fichier existant).

Contenu de fichiers

`cat` (concatener) : permet de concaténer et d'afficher des fichiers.

```
cat fich
cat fich1 fich2
```

`less` ou `more` : affiche des fichiers **page par page** 

.

```
less fich
```

`file` : permet d'identifier le type d'un fichier.

```
file fich
```

`mv` : déplace et/ou renomme un fichier ou un répertoire (*move*).

`vim` (console), `emacs`, `geany`, `gedit`, ... : editeurs de texte.

Copie de fichiers/répertoires

`cp` (copy) : permet de copier :

- des fichiers
- des répertoires avec l'option -**R**

Syntaxe : `cp` *source(s) destination*
source(s) = ce qui est copié
destination = vers où c'est copié

Exemple : `cp index.html /home/toto/`
Copie le fichier *index.html* dans le répertoire *toto* (si *toto* existe)

Attention :

- *source(s)* = fichiers à copier
- *source(s)* peut être des fichiers/répertoires
- *destination* = où on souhaite les copier
- *destination* peut exister ou non, et peut être un répertoire ou un fichier

Copie de fichiers/répertoires (exemple)

1. cp fich1 fich2
2. cp fich /rep
3. cp fich .
4. cp fich1 ... fichn /rep
5. cp -R rep1 rep2 (*rep2* existe ou non, copie dans *rep2* ou nommé *rep2*)

Renommage/déplacement de fichiers

`mv` (move) : permet de

- changer le nom (renommer) des fichiers ou répertoires,
- déplacer des fichiers ou répertoires (équivalent à une copie, suivie d'une suppression).

Syntaxe : `mv` *source(s) destination*

source(s) = ce qui est copié
destination = vers où c'est copié

Exemple :

- `mv index.html accueil.html` (Renommage)
- `mv index.html /home/site/` (Déplacement)
- `mv index.html /home/site/accueil.html` (Déplacement + Renommage)

Suppression de fichiers ou répertoires

`rm` (remove) : permet de supprimer des fichiers ou répertoires

Option `-R` pour supprimer un répertoire (et son contenu)

Attention : pas de corbeille !

Syntaxe :
Pour supprimer un fichier : `rm` *fich*
Pour supprimer un répertoire :

`rm -r` *rep*
`rmdir` *rep*

Exemple :

- `rm /home/site/index.html`
- `rm -r /home/site`

Faites très attention avec cette commande !
Pour être prudent : rm -i rep (demande confirmation avant suppression)

Plan

1. Introduction
2. Ligne de commande (shell)
3. Système de fichiers
4. Droits d'accès
5. Commandes de base
6. Métacaractères

Méta-caractères

Les **méta-caractères** sont des caractères spéciaux interprétés par le shell.

Lorsqu'ils sont utilisés dans une ligne de commande (**seuls** ou **en combinaison** avec d'autres caractères ou méta-caractères), l'argument qui les contient est **remplacé** par un ou plusieurs nom(s) de fichier **correspondant**.

- `*` : n'importe quelle séquence de zéro, un ou plusieurs caractère(s)
- `?` : exactement un caractère
- `~` : le répertoire home de l'utilisateur courant
- `[...]` : exactement un caractère appartenant à la séquence décrite, exemples :
  - `[xyz]` : un des caractères `x`, `y` ou `z`
  - `[^xyz]` : n'importe quel caractère autre que `x`, `y` ou `z`
  - `[0-9]` : n'importe quel chiffre décimal
  - `[a-zA-Z_]` : n'importe quelle lettre minuscule ou majuscule, ou l'underscore.

Métacaractères : exemples

Exemples :

- Lister tous les fichiers dont la **première lettre** est un **a**
  ls a*
- Déplacer **tous** les fichiers **.c** du répertoire courant vers le répertoire *source*
  mv *.c source
- Lister tous les fichiers nommés *tp* suivi d'**un et un seul caractère** (*tp1*, *tp2*, *tp3*, ... mais pas *tp*, *tp12*, *tp1.txt* ...)
  ls tp?
- Lister tous les fichiers dont la **première lettre** est **a**, **b** ou **c**
  ls [abc]* ou `ls [a-c]*`
- Lister tous les fichiers qui se **terminent** par **7**, **8** ou **9**
  ls *[7-9]
- Lister tous les fichiers qui n**e se terminent pas** par **6**, **7**, **8**, **9**
  ls *[!6-9] ou ls *[^6-9]

Échappement

- On appelle **échappement** (en anglais *escaping*) le mécanisme qui permet d'inhiber le comportement spécial d'un caractère.

- Le shell propose plusieurs mécanismes d'échappement :
  - les guillemets doubles (") ou simples (') pour **échapper toute une chaîne** ;
  - l'anti-slash (\) pour **échapper un unique caractère**.

Exemples :

```
echo Comment allez-vous \?
echo "* et ? sont des méta-caractères"
```

Pour conclure !

- Concepts clés :
  - Arborescence, racine du système de fichier, répertoire de connexion, répertoire de travail
  - Chemin absolu, chemin relatif
  - Fichier, répertoire, inode, liens (direct et symbolique)
  - Droits d'accès

- Commandes clés :
  - man, pwd, cd, ls, file
  - mkdir, ln, rm, rmdir, cp, mv, rm
  - chmod
  - `cat, less, more, touch, echo`

Merci de votre attention

